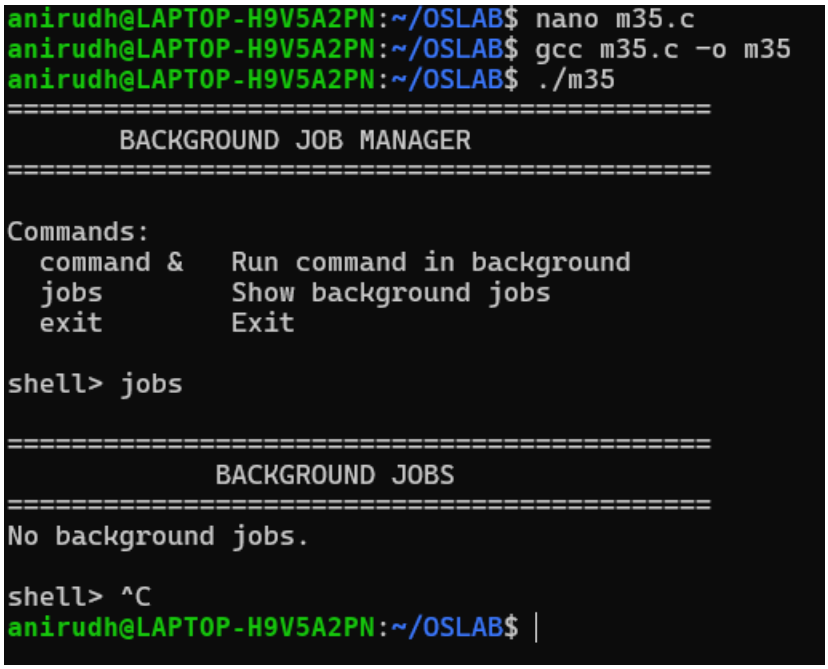

anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m35.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m35.c -o m35
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m35
BACKGROUND JOB MANAGER
Commands:
command & Run command in background
jobs Show background jobs
exit Exit
shell> jobs
BACKGROUND JOBS
No background jobs.
shell> ^C
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ cat m35.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 20
#define MAX_INPUT 256

typedef struct
{
    int job_id;
    pid_t pid;
    char command[MAX_INPUT];
    int running;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strncpy(
        jobs[job_count].command,
        command,
        MAX_INPUT - 1
    );

    jobs[job_count].command[MAX_INPUT - 1] = '\0';
    jobs[job_count].running = 1;

    printf("[%d] %d running  %s\n",
           jobs[job_count].job_id,
           jobs[job_count].pid,
           jobs[job_count].command);

    job_count++;
}

void check_background_jobs()
{
    int i;
    int status;
```

```
    for (i = 0; i < job_count; i++)
    {
        if (!jobs[i].running)
            continue;

        pid_t result =
            waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

        if (result == 0)
        {
            continue;
        }

        if (result == jobs[i].pid)
        {
            jobs[i].running = 0;

            if (WIFEXITED(status))
            {
                printf("\n[%d] Done  %s\n",
                       jobs[i].job_id,
                       jobs[i].command);
            }
            else
            {
                printf("\n[%d] Terminated  %s\n",
                       jobs[i].job_id,
                       jobs[i].command);
            }
        }
    }
}

void show_jobs()
{
    int i;

    check_background_jobs();

    printf("\n==========================================\n");
    printf("             BACKGROUND JOBS\n");
    printf("==========================================\n");

    if (job_count == 0)
    {
        printf("No background jobs.\n");
        return;
    }

    for (i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d  %s  %s\n",
```

```
                    ? "Running"
                    : "Completed",
                jobs[i].command);
    }
}

void launch_background_job(char *command)
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        char *args[32];
        int argc = 0;

        char *token =
            strtok(command, " \t\n");

        while (token != NULL &&
               argc < 31)
        {
            args[argc++] = token;
            token = strtok(NULL, " \t\n");
        }

        args[argc] = NULL;

        if (argc == 0)
            exit(EXIT_SUCCESS);

        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    add_job(pid, command);

    printf("Prompt returned immediately.\n");
}

int main()
{
    char input[MAX_INPUT];

    printf("========================================\n");
```

```
    printf("==========================================\n");

    printf("\nCommands:\n");
    printf("  command &   Run command in background\n");
    printf("  jobs        Show background jobs\n");
    printf("  exit        Exit\n");

    while (1)
    {
        check_background_jobs();

        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "jobs") == 0)
        {
            show_jobs();
            continue;
        }

        int length = strlen(input);

        if (length > 0 &&
            input[length - 1] == '&')
        {
            input[length - 1] = '\0';

            while (length > 1 &&
                   input[length - 2] == ' ')
            {
                input[length - 2] = '\0';
                length--;
            }

            launch_background_job(input);
        }
        else
        {
            printf("Use '&' to run a command in background.\n");
            printf("Example: sleep 10 &\n");
        }
    }

    printf("\nWaiting for remaining jobs...\n");
```

```
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            waitpid(
                jobs[i].pid,
                NULL,
                0
            );
        }
    }

    printf("All jobs completed.\n");

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m36.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m36.c -o m36
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m36
========================================
          JOB TABLE MANAGER
========================================

Commands:
  run <command>  Launch job
  jobs           Display jobs
  clean          Remove completed jobs
  exit           Exit

job-shell> l
Invalid command.

job-shell> -l
Invalid command.

job-shell> jobs

========================================
                JOB TABLE
========================================
No jobs.

job-shell> ^C
```

```
    job_table[job_count].state =
        JOB_RUNNING;

    printf(
        "[%d] PID=%d PGID=%d %s\n",
        job_table[job_count].job_id,
        pid,
        process_group,
        command
    );

    job_count++;
}

void update_job_states()
{
    int i;
    int status;

    for (i = 0; i < job_count; i++)
    {
        if (job_table[i].state ==
            JOB_COMPLETED)
        {
            continue;
        }

        pid_t result =
            waitpid(
                job_table[i].pid,
                &status,
                WNOHANG | WUNTRACED
            );

        if (result == 0)
            continue;

        if (result == job_table[i].pid)
        {
            if (WIFSTOPPED(status))
            {
                job_table[i].state =
                    JOB_STOPPED;
            }
            else if (WIFEXITED(status) ||
                     WIFSIGNALED(status))
            {
                job_table[i].state =
                    JOB_COMPLETED;
            }
        }
    }
}

void remove_completed_jobs()
{
    int i;
    int j;

    for (i = 0; i < job_count;)
    {
        if (job_table[i].state ==
            JOB_COMPLETED)
        {
            for (j = i;
                 j < job_count - 1;
                 j++)
            {
                job_table[j] =
                    job_table[j + 1];
            }

            job_count--;
        }
        else
        {
```

```
        }
    }
}

void display_jobs()
{
    int i;

    update_job_states();

    printf("\n==========================================\n");
    printf("               JOB TABLE\n");
    printf("==========================================\n");

    if (job_count == 0)
    {
        printf("No jobs.\n");
        return;
    }

    for (i = 0; i < job_count; i++)
    {
        printf(
            "[%d] PID=%d PGID=%d State=%s Command=%s\n",
            job_table[i].job_id,
            job_table[i].pid,
            job_table[i].process_group,
            state_name(job_table[i].state),
            job_table[i].command
        );
    }
}

void launch_job(char *command)
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        /*
         * Create a new process group.
         */
        setpgid(0, 0);

        char *args[32];
        int argc = 0;

        char *token =
            strtok(command, " \t\n");

        while (token != NULL &&
               argc < 31)
        {
            args[argc++] = token;

            token =
                strtok(NULL, " \t\n");
        }

        args[argc] = NULL;

        if (argc == 0)
            exit(EXIT_SUCCESS);

        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }
```

```
    add_job(
        pid,
        pid,
        command
    );
}

int main()
{
    char input[MAX_COMMAND];

    printf("============================================\n");
    printf("          JOB TABLE MANAGER\n");
    printf("============================================\n");

    printf("\nCommands:\n");
    printf("  run <command>  Launch job\n");
    printf("  jobs           Display jobs\n");
    printf("  clean          Remove completed jobs\n");
    printf("  exit           Exit\n");

    while (1)
    {
        update_job_states();

        printf("\njob-shell> ");
        fflush(stdout);

        if (fgets(
                input,
                sizeof(input),
                stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "jobs") == 0)
        {
            display_jobs();
            continue;
        }

        if (strcmp(input, "clean") == 0)
        {
            update_job_states();
            remove_completed_jobs();

            printf("Completed jobs removed.\n");
            continue;
        }

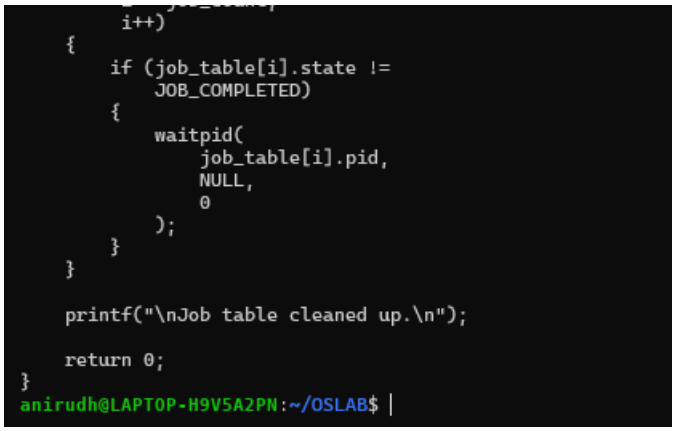
        if (strncmp(input, "run ", 4) == 0)
        {
            char command[MAX_COMMAND];

            strcpy(
                command,
                input + 4
            );

            launch_job(command);
            continue;
        }

        printf("Invalid command.\n");
    }

    /*
     * Wait for remaining jobs.
```



```
         i++)
    {
        if (job_table[i].state !=
            JOB_COMPLETED)
        {
            waitpid(
                job_table[i].pid,
                NULL,
                0
            );
        }
    }

    printf("\nJob table cleaned up.\n");

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```